

HiveMQ Blog Post

Integrating AI-Driven Computer Vision with a Unified Namespace: How HiveMQ Powers Real-Time Anomaly Detection

Implementing a Unified Namespace (UNS) with the HiveMQ MQTT broker enables centralized data flow, allowing diverse systems to both publish and access data seamlessly. At Coretecs, we leveraged this setup to develop a computer vision AI system that dynamically tunes its anomaly detection capabilities based on real-time process data. Detected anomalies are published to the MQTT broker, where other systems can access this data for various uses. In our case, we chose to historize detected defects and display live images on the current SCADA system for immediate insights.

In this post, we'll explore our approach to integrating AI with a UNS, showcasing how seamlessly data can flow across different systems.

What is a Unified Namespace (UNS) and Why Use HiveMQ?

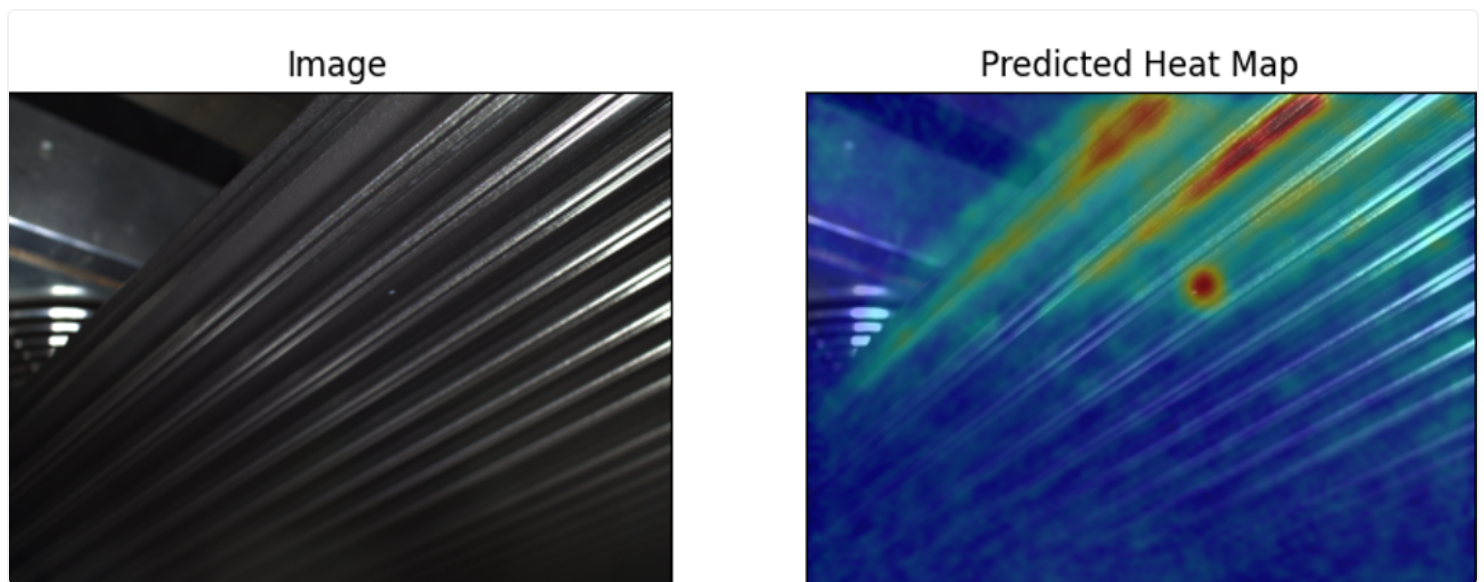
Upgrading a factory to meet Industry 4.0 standards often involves centralizing all available data, enabling flexibility and scalability as new systems and processes are added. A UNS provides the structure needed for this centralized data by creating a common namespace where data from all sources is accessible. This simplifies data management and enables non-experts to easily locate relevant data points.

To implement our UNS, we used the MQTT protocol with HiveMQ as the broker. MQTT is ideal for industrial settings due to its lightweight nature, and HiveMQ provides robust scalability, allowing thousands of devices and systems to publish and consume data simultaneously. With HiveMQ, our computer vision AI system could reliably publish detected anomalies, while other systems, such as the SCADA platform, consumed this data for real-time monitoring and historization.

For a deeper dive into UNS and its benefits in Industry 4.0, check out [HiveMQ's blog on Unified Namespace](#).

Designing the Anomaly Detection System with Computer Vision AI

Our primary goal was to create an anomaly detection model trained exclusively on anomaly-free images. This approach meant the model would label anything that deviated from the normal image state as a potential defect. However, it also introduced challenges: the model was sensitive to minor changes in lighting or camera positioning, which could lead to false positives. The model's performance is shown in the image below.



Using a Linear Tracker for Filtering

Since our system monitors a continuous extrusion process, defects detected at the start of the image frame should also appear later in the process as the material moves. By implementing a linear tracking algorithm, we could filter out false positives and track genuine defects as they progress through the image sequence.

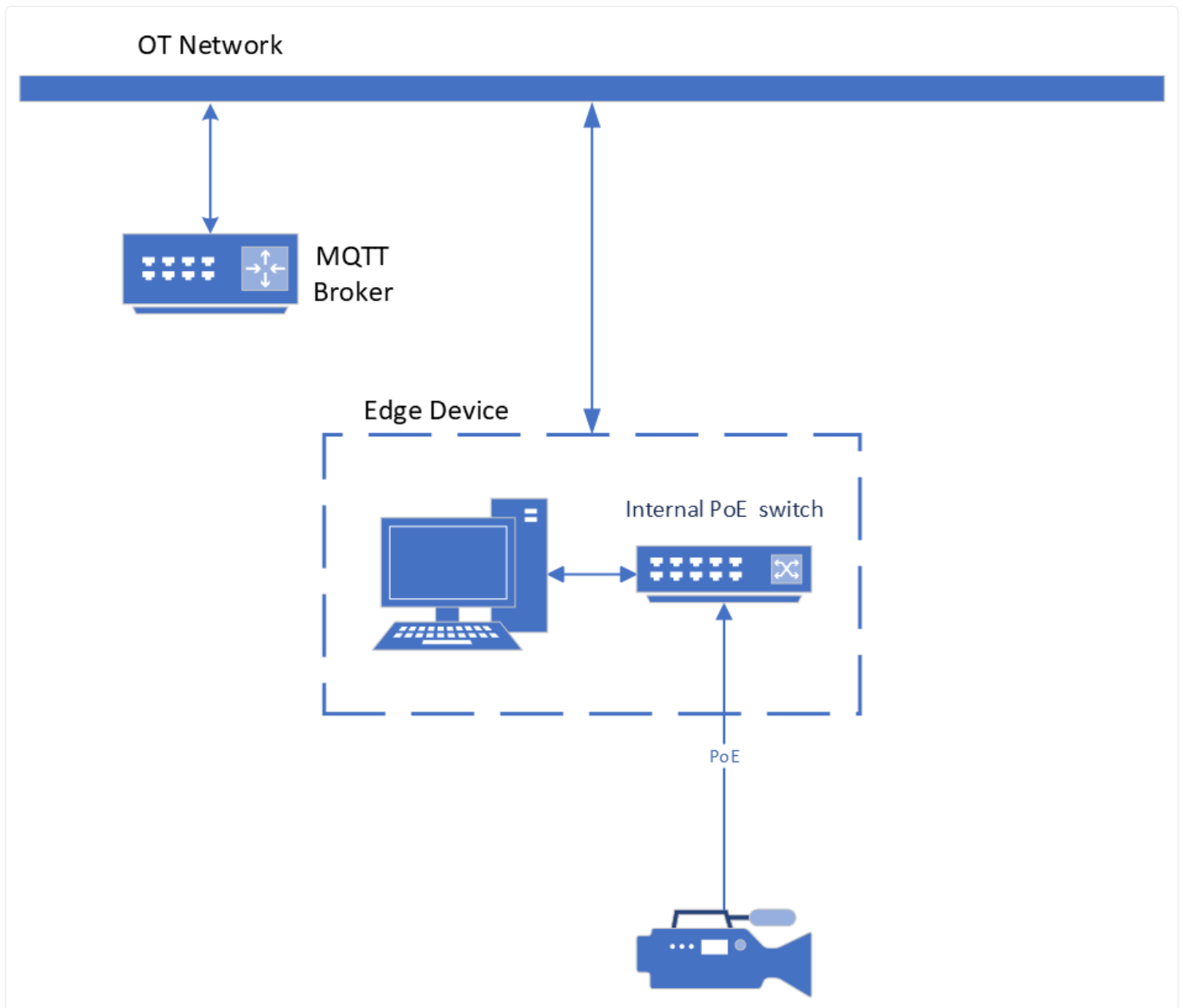
To enhance tracking accuracy, we accessed the extrusion speed in real-time via the HiveMQ MQTT broker, adjusting the tracker parameters accordingly. This dynamic adjustment allowed us to use a lightweight AI model that could handle occasional lighting changes while maintaining reliability. The model outputs a pixel-level anomaly score array, which we process to segment the defect areas using a threshold, then generate bounding boxes around these regions.

After filtering out false positives, the system publishes an image of the defect, along with metadata such as the anomaly score, and start and end times. An example of such an image can be seen below.



Integrating the AI System with HiveMQ and the UNS

The trained model was exported as an ONNX model and deployed in a Python environment on an edge device connected to the cameras and the MQTT broker. This setup enables the system to perform inference close to the data source, reducing latency and improving response time. A Flask server serves as a frontend, providing engineers with a user interface for managing inference settings and adjusting detection parameters as needed. The overall hardware setup is illustrated in the diagram below.



In addition to anomaly detection, the system continuously publishes a live feed from the camera to the MQTT broker, providing real-time visibility across the production line. The live image data, converted into a base64 string, is accessible to other systems within the UNS, including the SCADA platform, which displays ongoing production and any detected anomalies side by side. This live view offers engineers an integrated, real-time snapshot of production quality.

To ensure the accuracy of anomaly tracking, we read the extrusion speed from the MQTT broker in Python, continuously updating the linear tracker to filter false positives dynamically. When the model identifies a genuine defect, it captures the image with a bounding box around the defect, converts it into a base64 string, and publishes it along with the anomaly metadata to the MQTT broker. This metadata-rich payload ensures each defect's details are accessible for further analysis or visualization. Here's an example payload structure:

```

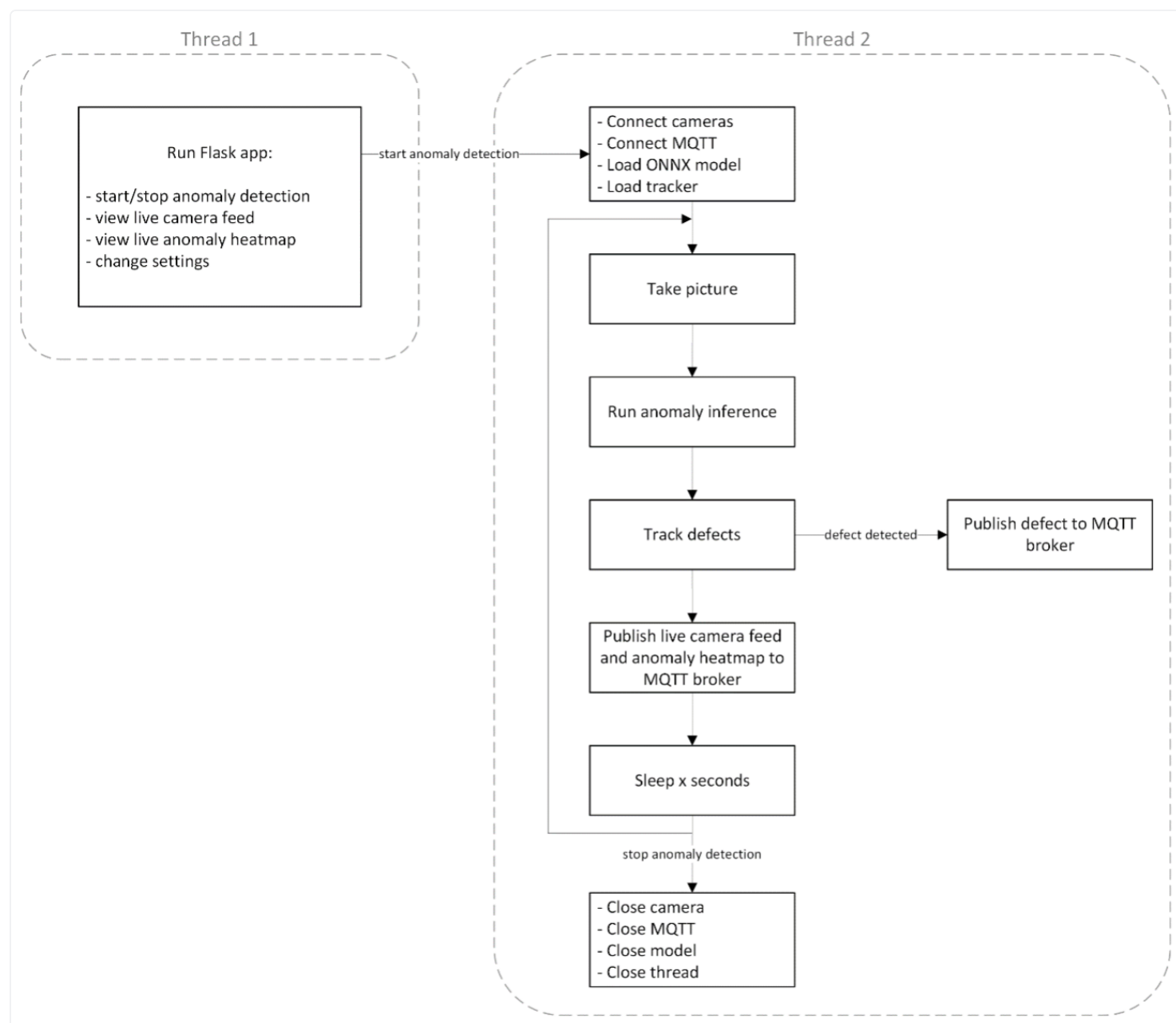
payload = {
  "metrics": [
    {
      "name": "IMAGE_DATA",
      "timestamp": timestamp,
      "datatype": "string",
      "value": encoded_image,
    },
    {
      "name": "SCORE",
      "timestamp": timestamp,
      "datatype": "float",
      "value": score,
    },
    {
      "name": "LABEL",
      "timestamp": timestamp,
      "datatype": "string",
      "value": label,
    },
    {
      "name": "START_TIME",
      "timestamp": timestamp,
      "datatype": "datetime",
      "value": start_time,
    },
    {
      "name": "END_TIME",
      "timestamp": timestamp,
      "datatype": "datetime",
      "value": end_time,
    },
    {
      "name": "TOTAL_TIME",
      "timestamp": timestamp,
      "datatype": "int",
      "value": end_time - start_time,
    },
  ]
}

```

This structured payload format ensures that each defect's data is easy to parse, consume, and display across the UNS, making it accessible to any subscribing systems for real-time insights or historical analysis.

Inference Workflow

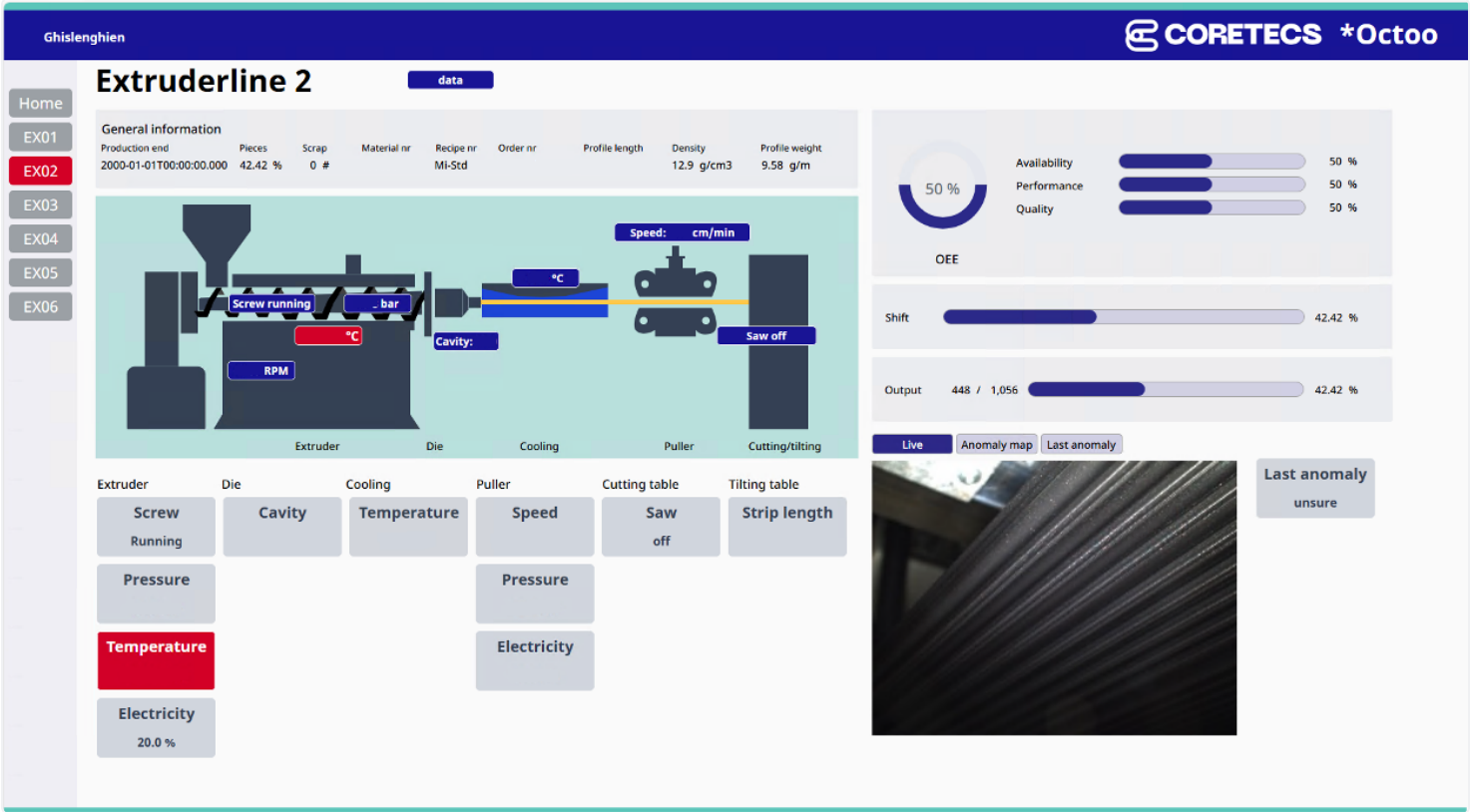
The anomaly detection system on the edge device operates with two primary components: a Flask web application and an anomaly detection thread. The Flask app serves as the user interface, enabling users to initiate or stop anomaly detection, monitor system status, and configure detection settings. When detection is activated, a separate thread runs in parallel to handle the inference tasks, ensuring efficient processing without interrupting the app's responsiveness. The workflow used during the inference phase is illustrated in the diagram below.



Visualization and Historization in the SCADA System

With our model publishing detected defects, we set up systems to historize and display them in real time on the SCADA platform. For historization, we used Canary to handle and store the base64 images. For SCADA visualization, we used Ignition, which displays the latest detected defect using a default component to decode and render the base64 image.

The live display of defect images enables engineers to monitor product quality at a glance, making the AI system a valuable tool in maintaining production standards (see screenshot below).



Conclusion

By integrating our AI-driven anomaly detection system with a Unified Namespace (UNS) and leveraging HiveMQ's MQTT broker, we've transformed data flow and enabled real-time defect detection within our manufacturing process. This setup not only centralizes and streamlines data sharing but also empowers engineers with immediate, actionable insights by historizing and visualizing detected anomalies on the SCADA system. This approach enhances quality control, ensures more responsive decision-making, and lays a solid foundation for future AI innovations and predictive analytics within our operations.